COMP102: DATA STRUCTURES WITH PYTHON

Python Foundations for Data Structures

Jalal Maaouni
March 13, 2026

UM6P University Mohammed VI Polytechnic
THE UM6P VANGUARD CENTER

#Empowering Minds.

Plan

1. Course overview
2. Why this course starts here
3. Python objects and mutability
4. Core built-in structures
5. Hidden costs
6. Hashing
7. Complexity and recap

Course overview

Course overview and lecture map

- This course studies how data is **represented**, **organized**, and **manipulated**.

Course overview and lecture map

- This course studies how data is **represented**, **organized**, and **manipulated**.
- We will move from Python built-ins to **stacks, queues, linked lists, trees, heaps, and graphs**.

Course overview and lecture map

- This course studies how data is **represented**, **organized**, and **manipulated**.
- We will move from Python built-ins to **stacks, queues, linked lists, trees, heaps, and graphs**.
- We will also study **searching, sorting, hashing, and complexity** as tools for choosing the right representation.

Course overview and lecture map

- This course studies how data is **represented**, **organized**, and **manipulated**.
- We will move from Python built-ins to **stacks, queues, linked lists, trees, heaps, and graphs**.
- We will also study **searching, sorting, hashing, and complexity** as tools for choosing the right representation.
- The course ends by connecting in-memory structures to **persistent storage** through SQL and NoSQL.

Course overview and lecture map

- This course studies how data is **represented**, **organized**, and **manipulated**.
- We will move from Python built-ins to **stacks, queues, linked lists, trees, heaps, and graphs**.
- We will also study **searching, sorting, hashing, and complexity** as tools for choosing the right representation.
- The course ends by connecting in-memory structures to **persistent storage** through SQL and NoSQL.

Lecture 1 lays the foundation: before building new structures, we need to understand how Python already stores and exposes data.

What is a data structure?

- A **data structure** is a way of organizing data so that some operations become natural and efficient.

What is a data structure?

- A **data structure** is a way of organizing data so that some operations become natural and efficient.
- The same information can be represented in several ways.

What is a data structure?

- A **data structure** is a way of organizing data so that some operations become natural and efficient.
- The same information can be represented in several ways.
- Different representations make different operations **easy**, **hard**, **fast**, or **wasteful**.

What is a data structure?

- A **data structure** is a way of organizing data so that some operations become natural and efficient.
- The same information can be represented in several ways.
- Different representations make different operations **easy**, **hard**, **fast**, or **wasteful**.

A data structure is not just a container. It is a **design choice** about representation, operations, and cost.

What is a data structure?

- A **data structure** is a way of organizing data so that some operations become natural and efficient.
- The same information can be represented in several ways.
- Different representations make different operations **easy**, **hard**, **fast**, or **wasteful**.

A data structure is not just a container. It is a **design choice** about representation, operations, and cost.

Examples of questions data structures answer

- Can I access the i-th element quickly?
- Can I test membership quickly?
- Can I preserve insertion order?
- Can I associate a key with a value?

Why this course starts here

What the course studies and how Lecture 1 fits into the whole arc

Why data structures matter in practice

- Suppose you must check whether a username already exists among one million users.

Why data structures matter in practice

- Suppose you must check whether a username already exists among one million users.
- Suppose you must preserve the order in which tasks were submitted.

Why data structures matter in practice

- Suppose you must check whether a username already exists among one million users.
- Suppose you must preserve the order in which tasks were submitted.
- Suppose you must map each student ID to a grade.

Why data structures matter in practice

- Suppose you must check whether a username already exists among one million users.
- Suppose you must preserve the order in which tasks were submitted.
- Suppose you must map each student ID to a grade.
- Suppose you must avoid duplicates while tracking visited pages or graph nodes.

Why data structures matter in practice

- Suppose you must check whether a username already exists among one million users.
- Suppose you must preserve the order in which tasks were submitted.
- Suppose you must map each student ID to a grade.
- Suppose you must avoid duplicates while tracking visited pages or graph nodes.

These are not the same problem. The right structure depends on the **operation you care about most**.

Learning goals for today

By the end of this lecture, you should be able to:

- explain what a Python object is, at least conceptually,

Learning goals for today

By the end of this lecture, you should be able to:

- explain what a Python object is, at least conceptually,
- distinguish **mutable** from **immutable** objects,

Learning goals for today

By the end of this lecture, you should be able to:

- explain what a Python object is, at least conceptually,
- distinguish **mutable** from **immutable** objects,
- choose between `list`, `tuple`, `dict`, and `set`,

Learning goals for today

By the end of this lecture, you should be able to:

- explain what a Python object is, at least conceptually,
- distinguish **mutable** from **immutable** objects,
- choose between `list`, `tuple`, `dict`, and `set`,
- explain why some operations are cheap on one structure and expensive on another,

Learning goals for today

By the end of this lecture, you should be able to:

- explain what a Python object is, at least conceptually,
- distinguish **mutable** from **immutable** objects,
- choose between `list`, `tuple`, `dict`, and `set`,
- explain why some operations are cheap on one structure and expensive on another,
- describe the basic intuition behind **slicing**, **hashing**, and **complexity**.

Learning goals for today

By the end of this lecture, you should be able to:

- explain what a Python object is, at least conceptually,
- distinguish **mutable** from **immutable** objects,
- choose between `list`, `tuple`, `dict`, and `set`,
- explain why some operations are cheap on one structure and expensive on another,
- describe the basic intuition behind **slicing**, **hashing**, and **complexity**.

Today is not a review of Python syntax. It is a first lesson in **representation-aware programming**.

Python is an object-based language

- In Python, values are **objects**, and classes define their behavior.

Python is an object-based language

- In Python, values are **objects**, and classes define their behavior.
- A variable name such as `x` or `data` is only an **identifier**.

Python is an object-based language

- In Python, values are **objects**, and classes define their behavior.
- A variable name such as `x` or `data` is only an **identifier**.
- The identifier does not store the value "inside itself"; it **refers** to an object.

Python is an object-based language

- In Python, values are **objects**, and classes define their behavior.
- A variable name such as `x` or `data` is only an **identifier**.
- The identifier does not store the value "inside itself"; it **refers** to an object.
- The object has a concrete type, such as `list`, `tuple`, `dict`, `set`, or `str`.

Python is an object-based language

- In Python, values are **objects**, and classes define their behavior.
- A variable name such as `x` or `data` is only an **identifier**.
- The identifier does not store the value "inside itself"; it **refers** to an object.
- The object has a concrete type, such as `list`, `tuple`, `dict`, `set`, or `str`.

In Python, the important question is not only "what value do I have?" but also "what **object** do I refer to, and what operations does its class support?"

Identifiers and assignment

Assignment binds a name to an object

```
data = [10, 20, 30]
```

Identifiers and assignment

Assignment binds a name to an object

```
data = [10, 20, 30]
```

- `data` is an **identifier**.

Identifiers and assignment

Assignment binds a name to an object

```
data = [10, 20, 30]
```

- `data` is an **identifier**.
- `[10, 20, 30]` creates a new `list` object.

Identifiers and assignment

Assignment binds a name to an object

```
data = [10, 20, 30]
```

- `data` is an **identifier**.
- `[10, 20, 30]` creates a new `list` object.
- The assignment statement binds the name `data` to that object.

Identifiers and assignment

Assignment binds a name to an object

```
data = [10, 20, 30]
```

- `data` is an **identifier**.
- `[10, 20, 30]` creates a new `list` object.
- The assignment statement binds the name `data` to that object.

Remark 2.1. Important comparison

The meaning is closer to a **reference** than to a box that physically contains the data.

Identifiers and assignment

Assignment binds a name to an object

```
data = [10, 20, 30]
```

- `data` is an **identifier**.
- `[10, 20, 30]` creates a new `list` object.
- The assignment statement binds the name `data` to that object.

Remark 2.1. Important comparison

The meaning is closer to a **reference** than to a box that physically contains the data.

Assignment in Python does not automatically imply copying.

Aliasing and rebinding are different

Aliasing

```
data = [10, 20, 30]
backup = data
```

Aliasing and rebinding are different

Aliasing

```
data = [10, 20, 30]
backup = data
```

- Now `data` and `backup` are two names for the **same list object**.

Aliasing and rebinding are different

Aliasing

```
data = [10, 20, 30]
backup = data
```

- Now `data` and `backup` are two names for the **same list object**.
- This is called an **alias**.

Aliasing and rebinding are different

Aliasing

```
data = [10, 20, 30]
backup = data
```

- Now `data` and `backup` are two names for the **same list object**.
- This is called an **alias**.

Rebinding

```
data = data + [40]
```

Aliasing and rebinding are different

Aliasing

```
data = [10, 20, 30]
backup = data
```

- Now `data` and `backup` are two names for the **same list object**.
- This is called an **alias**.

Rebinding

```
data = data + [40]
```

- The right-hand side is evaluated first.

Aliasing and rebinding are different

Aliasing

```
data = [10, 20, 30]
backup = data
```

- Now `data` and `backup` are two names for the **same list object**.
- This is called an **alias**.

Rebinding

```
data = data + [40]
```

- The right-hand side is evaluated first.
- A **new list object** is created.

Aliasing and rebinding are different

Aliasing

```
data = [10, 20, 30]
backup = data
```

- Now `data` and `backup` are two names for the **same list object**.
- This is called an **alias**.

Rebinding

```
data = data + [40]
```

- The right-hand side is evaluated first.
- A **new list object** is created.
- Then `data` is rebound to that new object.

Creating objects: literals, constructors, and methods

Three common ways objects appear

```
tags = {"python", "graphs"}       # literal
name_chars = list("ada")          # constructor
tasks = ["read", "code", "rest"]
tasks.append("submit")            # method call on an object
```

Creating objects: literals, constructors, and methods

Three common ways objects appear

```
tags = {"python", "graphs"}       # literal
name_chars = list("ada")          # constructor
tasks = ["read", "code", "rest"]
tasks.append("submit")            # method call on an object
```

- A **literal** directly describes a new object.

Creating objects: literals, constructors, and methods

Three common ways objects appear

```
tags = {"python", "graphs"}       # literal
name_chars = list("ada")          # constructor
tasks = ["read", "code", "rest"]
tasks.append("submit")            # method call on an object
```

- A **literal** directly describes a new object.
- A **constructor** creates an object from existing data.

Creating objects: literals, constructors, and methods

Three common ways objects appear

```
tags = {"python", "graphs"}       # literal
name_chars = list("ada")          # constructor
tasks = ["read", "code", "rest"]
tasks.append("submit")            # method call on an object
```

- A **literal** directly describes a new object.
- A **constructor** creates an object from existing data.
- A **method** is a behavior attached to a specific object.

Creating objects: literals, constructors, and methods

Three common ways objects appear

```
tags = {"python", "graphs"}        # literal
name_chars = list("ada")           # constructor
tasks = ["read", "code", "rest"]
tasks.append("submit")             # method call on an object
```

- A **literal** directly describes a new object.
- A **constructor** creates an object from existing data.
- A **method** is a behavior attached to a specific object.

Remark 2.2. Mutators vs accessors

Some methods **change** the object, like `append`; others only **inspect** or return information.

Python objects and mutability

Representation determines operations and cost

Variables, objects, and references

Example

```
x = [1, 2, 3]
y = x
```

Variables, objects, and references

Example

```
x = [1, 2, 3]
y = x
```

- There are **two variable names**: `x` and `y`.
- There is only **one list object** so far.

Variables, objects, and references

Example

```
x = [1, 2, 3]
y = x
```

- There are **two variable names**: `x` and `y`.
- There is only **one list object** so far.
- The assignment `y = x` creates another **reference** to the same object.

In Python, variables do not behave like little boxes that copy contents automatically. They refer to objects.

Variables, objects, and references

Example

```
x = [1, 2, 3]
y = x
```

- There are **two variable names**: `x` and `y`.
- There is only **one list object** so far.
- The assignment `y = x` creates another **reference** to the same object.

In Python, variables do not behave like little boxes that copy contents automatically. They refer to objects.

Remark 3.4. Question to ask yourself

If we now execute `y.append(4)`, should `x` change too?

Mutability through aliasing

A common surprise

```
a = [1, 2]
b = a
b.append(3)
print(a)
```

Mutability through aliasing

A common surprise

```
a = [1, 2]
b = a
b.append(3)
print(a)
```

- The method `append` changes the list **in place**.
- Since `a` and `b` refer to the same object, the mutation is visible through both names.

Mutability through aliasing

A common surprise

```
a = [1, 2]
b = a
b.append(3)
print(a)
```

- The method `append` changes the list **in place**.
- Since `a` and `b` refer to the same object, the mutation is visible through both names.

Remark 3.7. Common beginner mistake

Students often expect `b = a` to create an independent copy. It does not.

Mutability through aliasing

A common surprise

```
a = [1, 2]
b = a
b.append(3)
print(a)
```

- The method `append` changes the list **in place**.
- Since `a` and `b` refer to the same object, the mutation is visible through both names.

Remark 3.8. Common beginner mistake

Students often expect `b = a` to create an independent copy. It does not.

Mutability matters because shared references create **side effects**. A change through one name can affect another part of the program.

Mutable vs immutable objects

Table: Some common Python types

Type	Mutable?	What a "change" means	Typical use
`int`	No	a new integer object is produced	counters, sizes
`str`	No	a new string object is produced	text, labels
`tuple`	No	a new tuple object is produced	fixed records, coordinates
`list`	Yes	the same list can be modified in place	ordered collections
`dict`	Yes	key/value pairs can be added or updated	mappings, indexes
`set`	Yes	elements can be added or removed	uniqueness, fast membership

Core built-in structures

Why simple Python code can surprise you

Lists and tuples

`list`

- ordered sequence,

Lists and tuples

`list`

- ordered sequence,
- mutable,

Lists and tuples

`list`

- ordered sequence,
- mutable,
- supports indexing,

`tuple`

Lists and tuples

`list`

- ordered sequence,
- mutable,
- supports indexing,
- good when the collection evolves over time.

`tuple`

Lists and tuples

`list`

- ordered sequence,
- mutable,
- supports indexing,
- good when the collection evolves over time.

`tuple`

- ordered sequence,

Lists and tuples

`list`

- ordered sequence,
- mutable,
- supports indexing,
- good when the collection evolves over time.

`tuple`

- ordered sequence,
- immutable,

Lists and tuples

`list`

- ordered sequence,
- mutable,
- supports indexing,
- good when the collection evolves over time.

`tuple`

- ordered sequence,
- immutable,
- supports indexing,

Lists and tuples

`list`

- ordered sequence,
- mutable,
- supports indexing,
- good when the collection evolves over time.

`tuple`

- ordered sequence,
- immutable,
- supports indexing,
- good when the record has a fixed meaning.

Lists and tuples

Use a tuple when you want to say: **this structure has a fixed shape**. Use a list when you want to say: **this collection will grow, shrink, or change**.

Lists and tuples

Use a tuple when you want to say: **this structure has a fixed shape**. Use a list when you want to say: **this collection will grow, shrink, or change**.

Example 4.2. Typical examples

A shopping cart or task list naturally fits a `list`; a point like `(x, y)` naturally fits a `tuple`.

Dictionaries and sets

`dict`

- stores **key** $\rightarrow$ **value** pairs,

Dictionaries and sets

`dict`

- stores **key** $\rightarrow$ **value** pairs,
- natural when one piece of data identifies another,

Dictionaries and sets

`dict`

- stores **key** $\rightarrow$ **value** pairs,
- natural when one piece of data identifies another,
- average membership and lookup are very fast.

`set`

Dictionaries and sets

`dict`

- stores **key** $\rightarrow$ **value** pairs,
- natural when one piece of data identifies another,
- average membership and lookup are very fast.

`set`

- stores unique elements,

Dictionaries and sets

`dict`

- stores **key** $\rightarrow$ **value** pairs,
- natural when one piece of data identifies another,
- average membership and lookup are very fast.

`set`

- stores unique elements,
- ideal for membership testing,

Dictionaries and sets

`dict`

- stores **key** $\rightarrow$ **value** pairs,
- natural when one piece of data identifies another,
- average membership and lookup are very fast.

`set`

- stores unique elements,
- ideal for membership testing,
- natural when duplicates should disappear.

Example 4.3. Typical examples

Use a `dict` for `student_id` $\rightarrow$ `grade`. Use a `set` for `visited_nodes` or unique email addresses.

Example 4.4. Typical examples

Use a `dict` for `student_id` $\rightarrow$ `grade`. Use a `set` for `visited_nodes` or unique email addresses.

Lists and tuples answer questions about **position**. Dictionaries and sets answer questions about **identity** and **membership**.

Choosing the right structure (1/2)

1. You must preserve insertion order and allow duplicates.

Suggested structure

Choosing the right structure (1/2)

1. You must preserve insertion order and allow duplicates.

Suggested structure

- **1** $\rightarrow$ **list**: order matters and duplicates are allowed.

Choosing the right structure (1/2)

1. You must preserve insertion order and allow duplicates.
2. You must store unique course tags such as `python`, `algorithms`, `graphs`.

Suggested structure

- **1** $\rightarrow$ **list**: order matters and duplicates are allowed.

Choosing the right structure (1/2)

1. You must preserve insertion order and allow duplicates.
2. You must store unique course tags such as `python`, `algorithms`, `graphs`.

Suggested structure

- **1** $\rightarrow$ **list**: order matters and duplicates are allowed.
- **2** $\rightarrow$ **set**: uniqueness is the core requirement.

Choosing the right structure (1/2)

1. You must preserve insertion order and allow duplicates.
2. You must store unique course tags such as `python`, `algorithms`, `graphs`.
3. You must map each student ID to a GPA.

Suggested structure

- **1** $\rightarrow$ **list**: order matters and duplicates are allowed.
- **2** $\rightarrow$ **set**: uniqueness is the core requirement.

Choosing the right structure (1/2)

1. You must preserve insertion order and allow duplicates.
2. You must store unique course tags such as `python`, `algorithms`, `graphs`.
3. You must map each student ID to a GPA.

Suggested structure

- **1** $\rightarrow$ **list**: order matters and duplicates are allowed.
- **2** $\rightarrow$ **set**: uniqueness is the core requirement.
- **3** $\rightarrow$ **dict**: one key identifies one associated value.

Choosing the right structure (1/2)

1. You must preserve insertion order and allow duplicates.
2. You must store unique course tags such as `python`, `algorithms`, `graphs`.
3. You must map each student ID to a GPA.

Suggested structure

- **1** $\rightarrow$ **list**: order matters and duplicates are allowed.
- **2** $\rightarrow$ **set**: uniqueness is the core requirement.
- **3** $\rightarrow$ **dict**: one key identifies one associated value.

Always start from the **dominant operation or constraint**: order, indexing, mapping, or uniqueness.

Choosing the right structure (2/2)

1. You must represent a fixed RGB color like `(255, 128, 0)`.

Suggested structure

Choosing the right structure (2/2)

1. You must represent a fixed RGB color like `(255, 128, 0)`.

Suggested structure

- **1** → **tuple**: the record has a fixed size and fixed meaning.

Choosing the right structure (2/2)

1. You must represent a fixed RGB color like `(255, 128, 0)`.
2. You must track whether a page or graph node has already been visited.

Suggested structure

- **1** → **tuple**: the record has a fixed size and fixed meaning.

Choosing the right structure (2/2)

1. You must represent a fixed RGB color like `(255, 128, 0)`.
2. You must track whether a page or graph node has already been visited.

Suggested structure

- **1** → **tuple**: the record has a fixed size and fixed meaning.
- **2** → **set**: fast membership is the key need.

Choosing the right structure (2/2)

1. You must represent a fixed RGB color like `(255, 128, 0)`.
2. You must track whether a page or graph node has already been visited.

Suggested structure

- **1** → **tuple**: the record has a fixed size and fixed meaning.
- **2** → **set**: fast membership is the key need.

Remark 4.5. Mental rule

When you hesitate, do not ask "what data do I have?" first. Ask "what operations will I perform repeatedly?"

Hidden costs

Each built-in type answers a different need

Slicing is not free

Example

```
a = list(range(10))
b = a[2:7]
```

Slicing is not free

Example

```
a = list(range(10))
b = a[2:7]
```

- The expression `a[2:7]` does **not** mean "just remember the interval from 2 to 6".
- It creates a **new list object**.
- The elements are copied into that new list as references.

Slicing is not free

Example

```
a = list(range(10))
b = a[2:7]
```

- The expression `a[2:7]` does **not** mean "just remember the interval from 2 to 6".
- It creates a **new list object**.
- The elements are copied into that new list as references.

Beautiful syntax can still do real work. Slicing is convenient, but it is not a zero-cost view.

Slicing is not free

Example

```
a = list(range(10))
b = a[2:7]
```

- The expression `a[2:7]` does **not** mean "just remember the interval from 2 to 6".
- It creates a **new list object**.
- The elements are copied into that new list as references.

Beautiful syntax can still do real work. Slicing is convenient, but it is not a zero-cost view.

Remark 5.4. Why this matters later

Inside loops, recursion, or divide-and-conquer code, repeated slicing can quietly add significant overhead.

Other operations that may look cheaper than they are

- Testing membership in a `list` may require scanning many elements.

Other operations that may look cheaper than they are

- Testing membership in a `list` may require scanning many elements.
- Concatenating lists creates a new list rather than modifying both operands magically.

Other operations that may look cheaper than they are

- Testing membership in a `list` may require scanning many elements.
- Concatenating lists creates a new list rather than modifying both operands magically.
- Inserting near the beginning of a list may force many elements to shift.

Other operations that may look cheaper than they are

- Testing membership in a `list` may require scanning many elements.
- Concatenating lists creates a new list rather than modifying both operands magically.
- Inserting near the beginning of a list may force many elements to shift.
- Copy-like syntax often hides time and memory costs.

Other operations that may look cheaper than they are

- Testing membership in a `list` may require scanning many elements.
- Concatenating lists creates a new list rather than modifying both operands magically.
- Inserting near the beginning of a list may force many elements to shift.
- Copy-like syntax often hides time and memory costs.

A useful habit in this course: whenever Python syntax looks very convenient, ask yourself what work the interpreter must actually perform.

Hashing

Elegant syntax can hide real work

Mathematical intuition for hashing

Definition 6.1 Hashing intuition

A hash function takes an object and computes a numerical code that helps decide **where** that object should be stored or searched for inside a hash-based structure.

Mathematical intuition for hashing

Definition 6.2 Hashing intuition

A hash function takes an object and computes a numerical code that helps decide **where** that object should be stored or searched for inside a hash-based structure.

A simple abstract picture is:

$$h : U \longrightarrow \{0, 1, \ldots, m-1\}$$

Mathematical intuition for hashing

Definition 6.3 Hashing intuition

A hash function takes an object and computes a numerical code that helps decide **where** that object should be stored or searched for inside a hash-based structure.

A simple abstract picture is:

$$h : U \longrightarrow \{0, 1, \ldots, m-1\}$$

- **Deterministic:** the same object must always produce the same hash during a run.

Mathematical intuition for hashing

Definition 6.4 Hashing intuition

A hash function takes an object and computes a numerical code that helps decide **where** that object should be stored or searched for inside a hash-based structure.

A simple abstract picture is:

$$h : U \longrightarrow \{0, 1, \ldots, m-1\}$$

- **Deterministic:** the same object must always produce the same hash during a run.
- **Compatible with equality:** if two objects are equal, their hashes must also be equal.

Mathematical intuition for hashing

Definition 6.5 Hashing intuition

A hash function takes an object and computes a numerical code that helps decide **where** that object should be stored or searched for inside a hash-based structure.

A simple abstract picture is:

$$h : U \longrightarrow \{0, 1, \ldots, m-1\}$$

- **Deterministic:** the same object must always produce the same hash during a run.
- **Compatible with equality:** if two objects are equal, their hashes must also be equal.
- **Fast to compute:** otherwise lookup loses its advantage.

Mathematical intuition for hashing

Definition 6.6 Hashing intuition

A hash function takes an object and computes a numerical code that helps decide **where** that object should be stored or searched for inside a hash-based structure.

A simple abstract picture is:

$$h : U \longrightarrow \{0, 1, \ldots, m-1\}$$

- **Deterministic:** the same object must always produce the same hash during a run.
- **Compatible with equality:** if two objects are equal, their hashes must also be equal.
- **Fast to compute:** otherwise lookup loses its advantage.
- **Well spread:** many different objects should not collapse into the same bucket too often.

Collisions and why hash quality matters

- A **collision** happens when two different objects produce the same hash value.

Collisions and why hash quality matters

- A **collision** happens when two different objects produce the same hash value.
- Collisions do not make hashing useless; they make lookup less clean and potentially less fast.

Collisions and why hash quality matters

- A **collision** happens when two different objects produce the same hash value.
- Collisions do not make hashing useless; they make lookup less clean and potentially less fast.
- Good hash functions reduce clustering and help keep average lookup close to constant time.

Collisions and why hash quality matters

- A **collision** happens when two different objects produce the same hash value.
- Collisions do not make hashing useless; they make lookup less clean and potentially less fast.
- Good hash functions reduce clustering and help keep average lookup close to constant time.

Hashing does not promise perfection. It promises a very efficient **average-case organization** of data when hashes are well behaved.

Hashing and hashability in Python

What fails here?

```python
d = {}
d[[1, 2]] = "hello"
```

Hashing and hashability in Python

What fails here?

```python
d = {}
d[[1, 2]] = "hello"
```

- Dictionaries and sets rely on hashing.
- Their keys or elements must therefore be **hashable**.
- Lists are mutable, so their content can change after insertion.

Hashing and hashability in Python

What fails here?

```python
d = {}
d[[1, 2]] = "hello"
```

- Dictionaries and sets rely on hashing.
- Their keys or elements must therefore be **hashable**.
- Lists are mutable, so their content can change after insertion.

Remark 6.3. Why Python rejects the list key

If a key could change after being stored, the structure could no longer trust the original hash-based placement of that key.

Hashing and hashability in Python

What fails here?

```python
d = {}
d[[1, 2]] = "hello"
```

- Dictionaries and sets rely on hashing.
- Their keys or elements must therefore be **hashable**.
- Lists are mutable, so their content can change after insertion.

Remark 6.4. Why Python rejects the list key

If a key could change after being stored, the structure could no longer trust the original hash-based placement of that key.

As a first rule of thumb: immutable objects such as strings, numbers, and many tuples are natural candidates for hash-based use; mutable containers such as lists are not.

Complexity and recap

From mathematical idea to Python behavior

What complexity tries to measure

- Complexity asks how the **cost** of an operation or algorithm grows as the input size grows.

What complexity tries to measure

- Complexity asks how the **cost** of an operation or algorithm grows as the input size grows.
- The two most common costs are **time** and **space**.

What complexity tries to measure

- Complexity asks how the **cost** of an operation or algorithm grows as the input size grows.
- The two most common costs are **time** and **space**.
- We usually do not care about the exact number of milliseconds on one machine.

What complexity tries to measure

- Complexity asks how the **cost** of an operation or algorithm grows as the input size grows.
- The two most common costs are **time** and **space**.
- We usually do not care about the exact number of milliseconds on one machine.
- We care about the **growth pattern**: constant, linear, quadratic, logarithmic, and so on.

What complexity tries to measure

- Complexity asks how the **cost** of an operation or algorithm grows as the input size grows.
- The two most common costs are **time** and **space**.
- We usually do not care about the exact number of milliseconds on one machine.
- We care about the **growth pattern**: constant, linear, quadratic, logarithmic, and so on.

Complexity is about **scaling behavior**, not about stopwatch timing on a single laptop.

Big-O notation: first intuition

Definition 7.1 Informal Big-O idea

When we say an operation is in $O(g(n))$, we mean that for large inputs, its cost grows **no faster than a constant multiple** of $g(n)$ up to lower-order details.

Big-O notation: first intuition

Definition 7.2 Informal Big-O idea

When we say an operation is in $O(g(n))$, we mean that for large inputs, its cost grows **no faster than a constant multiple** of $g(n)$ up to lower-order details.

- $O(1)$ means the cost stays bounded as input grows.

Big-O notation: first intuition

Definition 7.3 Informal Big-O idea

When we say an operation is in $O(g(n))$, we mean that for large inputs, its cost grows **no faster than a constant multiple** of $g(n)$ up to lower-order details.

- $O(1)$ means the cost stays bounded as input grows.
- $O(n)$ means cost grows proportionally to the input size.

Big-O notation: first intuition

Definition 7.4 Informal Big-O idea

When we say an operation is in $O(g(n))$, we mean that for large inputs, its cost grows **no faster than a constant multiple** of $g(n)$ up to lower-order details.

- $O(1)$ means the cost stays bounded as input grows.
- $O(n)$ means cost grows proportionally to the input size.
- $O(n^2)$ means cost grows much faster as the input becomes large.

Big-O notation: first intuition

Definition 7.5 Informal Big-O idea

When we say an operation is in $O(g(n))$, we mean that for large inputs, its cost grows **no faster than a constant multiple** of $g(n)$ up to lower-order details.

- $O(1)$ means the cost stays bounded as input grows.
- $O(n)$ means cost grows proportionally to the input size.
- $O(n^2)$ means cost grows much faster as the input becomes large.

Remark 7.5. How to read Big-O in this lecture

Use it as a language for comparing growth classes, not as a claim about exact runtime.

Best case, average case, worst case

Table: Kinds of complexity

Case	Meaning	Simple intuition
Best case	the most favorable input configuration	the item is found immediately
Average case	the expected or typical cost over many inputs	the item is found after a moderate amount of work
Worst case	the maximum cost among inputs of size n	the structure must inspect everything relevant

Best case, average case, worst case

Table: Kinds of complexity

Case	Meaning	Simple intuition
Best case	the most favorable input configuration	the item is found immediately
Average case	the expected or typical cost over many inputs	the item is found after a moderate amount of work
Worst case	the maximum cost among inputs of size n	the structure must inspect everything relevant

- Best case can be reassuring, but it may be misleading if it rarely happens.

Best case, average case, worst case

Table: Kinds of complexity

Case	Meaning	Simple intuition
Best case	the most favorable input configuration	the item is found immediately
Average case	the expected or typical cost over many inputs	the item is found after a moderate amount of work
Worst case	the maximum cost among inputs of size n	the structure must inspect everything relevant

- Best case can be reassuring, but it may be misleading if it rarely happens.
- Worst case gives a safety guarantee.

Best case, average case, worst case

Table: Kinds of complexity

Case	Meaning	Simple intuition
Best case	the most favorable input configuration	the item is found immediately
Average case	the expected or typical cost over many inputs	the item is found after a moderate amount of work
Worst case	the maximum cost among inputs of size n	the structure must inspect everything relevant

- Best case can be reassuring, but it may be misleading if it rarely happens.
- Worst case gives a safety guarantee.
- Average case is often the most practical description for well-behaved hash-based structures.

Complexity intuition across common Python structures

Table: Typical cost intuition

Operation	list	tuple	set	dict
Indexing by position	$O(1)$	$O(1)$	—	—
Membership test	$O(n)$	$O(n)$	avg. $O(1)$	avg. $O(1)$ on keys
Append / add	append avg. $O(1)$	—	avg. $O(1)$	assignment avg. $O(1)$
Update by key / position	by index $O(1)$	—	—	avg. $O(1)$ by key

Complexity intuition across common Python structures

Table: Typical cost intuition

Operation	list	tuple	set	dict
Indexing by position	$O(1)$	$O(1)$	—	—
Membership test	$O(n)$	$O(n)$	avg. $O(1)$	avg. $O(1)$ on keys
Append / add	append avg. $O(1)$	—	avg. $O(1)$	assignment avg. $O(1)$
Update by key / position	by index $O(1)$	—	—	avg. $O(1)$ by key

- Lists and tuples are naturally strong for **ordered, positional access**.

Complexity intuition across common Python structures

Table: Typical cost intuition

Operation	list	tuple	set	dict
Indexing by position	$O(1)$	$O(1)$	—	—
Membership test	$O(n)$	$O(n)$	avg. $O(1)$	avg. $O(1)$ on keys
Append / add	append avg. $O(1)$	—	avg. $O(1)$	assignment avg. $O(1)$
Update by key / position	by index $O(1)$	—	—	avg. $O(1)$ by key

- Lists and tuples are naturally strong for **ordered, positional access**.
- Sets and dictionaries are naturally strong for **identity, membership, and key-based lookup**.

Complexity intuition across common Python structures

Table: Typical cost intuition

Operation	list	tuple	set	dict
Indexing by position	$O(1)$	$O(1)$	—	—
Membership test	$O(n)$	$O(n)$	avg. $O(1)$	avg. $O(1)$ on keys
Append / add	append avg. $O(1)$	—	avg. $O(1)$	assignment avg. $O(1)$
Update by key / position	by index $O(1)$	—	—	avg. $O(1)$ by key

- Lists and tuples are naturally strong for **ordered, positional access**.
- Sets and dictionaries are naturally strong for **identity, membership, and key-based lookup**.

Recap: the core ideas of Lecture 1

- Python variables refer to objects.

Recap: the core ideas of Lecture 1

- Python variables refer to objects.
- Mutability changes how safe it is to share and modify a structure.

Recap: the core ideas of Lecture 1

- Python variables refer to objects.
- Mutability changes how safe it is to share and modify a structure.
- `list`, `tuple`, `dict`, and `set` each answer a different family of needs.

Recap: the core ideas of Lecture 1

- Python variables refer to objects.
- Mutability changes how safe it is to share and modify a structure.
- `list`, `tuple`, `dict`, and `set` each answer a different family of needs.
- Slicing and other convenient operations may hide real work.

Recap: the core ideas of Lecture 1

- Python variables refer to objects.
- Mutability changes how safe it is to share and modify a structure.
- `list`, `tuple`, `dict`, and `set` each answer a different family of needs.
- Slicing and other convenient operations may hide real work.
- Hashing explains why dictionaries and sets are usually fast for lookup.

Recap: the core ideas of Lecture 1

- Python variables refer to objects.
- Mutability changes how safe it is to share and modify a structure.
- `list`, `tuple`, `dict`, and `set` each answer a different family of needs.
- Slicing and other convenient operations may hide real work.
- Hashing explains why dictionaries and sets are usually fast for lookup.
- Complexity gives us the language to compare those choices systematically.

Recap: the core ideas of Lecture 1

- Python variables refer to objects.
- Mutability changes how safe it is to share and modify a structure.
- `list`, `tuple`, `dict`, and `set` each answer a different family of needs.
- Slicing and other convenient operations may hide real work.
- Hashing explains why dictionaries and sets are usually fast for lookup.
- Complexity gives us the language to compare those choices systematically.

Main message: choosing a data structure means choosing which operations become natural, safe, and efficient.